

按键控制

按键控制

- 一、学习目标
- 二、硬件搭建
 - 独立按键原理
 - 按键消抖
- 三、实验步骤
 - 1. 打开SYSCONFIG配置工具
 - 2. 配置引脚模式
 - 3. 写入程序
 - 4. 编译
- 四、程序分析
- 五、实验现象

一、学习目标

- 1.学习MSPM0G3507主板的按键基本工作原理。
- 2.使用板载按键控制板载led灯的开关。

二、硬件搭建

独立按键原理

独立按键通常通过机械开关实现，其硬件结构由按键、连接线路、上拉/下拉电阻组成。按键闭合时，线路接通，信号电平发生变化；按键释放时，线路断开，信号恢复初始状态。

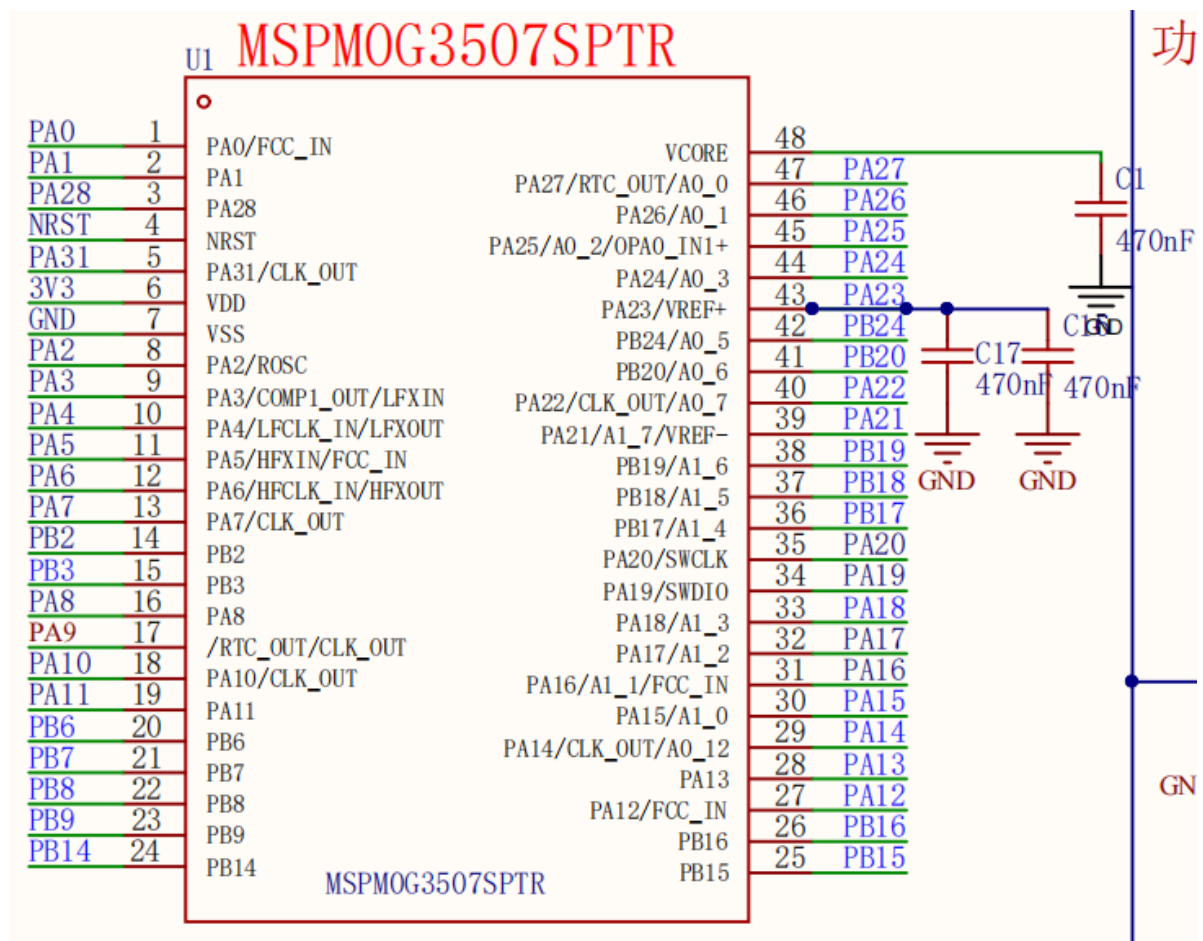
按键消抖

按键的机械弹性会导致按键在闭合或释放时，信号会在短时间内多次跳变，这被称为按键抖动。通常通过软件延时去抖或硬件加RC滤波电路来消除抖动。

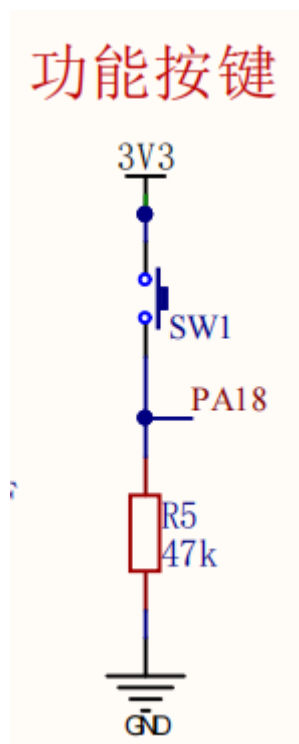
本次课程无需额外的硬件设备，直接利用MSPM0G3507主板上的板载功能按键。

在开发板的原理图中，PA18 引脚接入了一个按键，按键的一端连接高电平（3.3V），另一端连接到GPIO 引脚，同时通过一个 47kΩ 的下拉电阻 R5 将 PA18 引脚连接到地。当按键未按下时，由于下拉电阻的作用，PA18 引脚电平被拉低至 0V；当按键按下时，PA18 引脚直接接入高电平 3.3V，从而实现高低电平的变化。开发板通过检测 PA18 引脚的电平状态即可判断按键是否被按下。这种设计简单且稳定，能够有效避免引脚悬空导致的信号干扰问题。

MSPM0G3507主控图



按键部分原理图

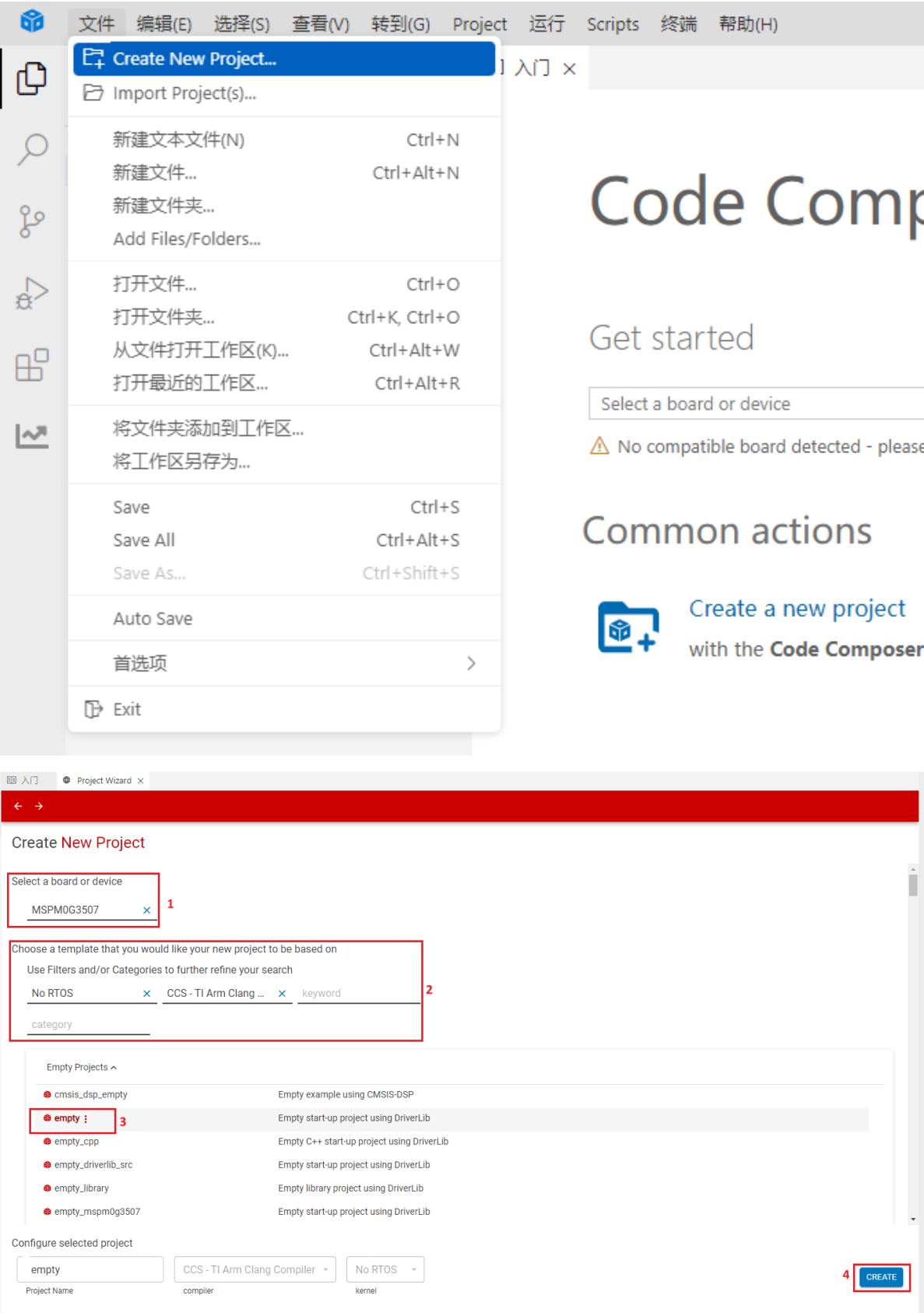


三、实验步骤

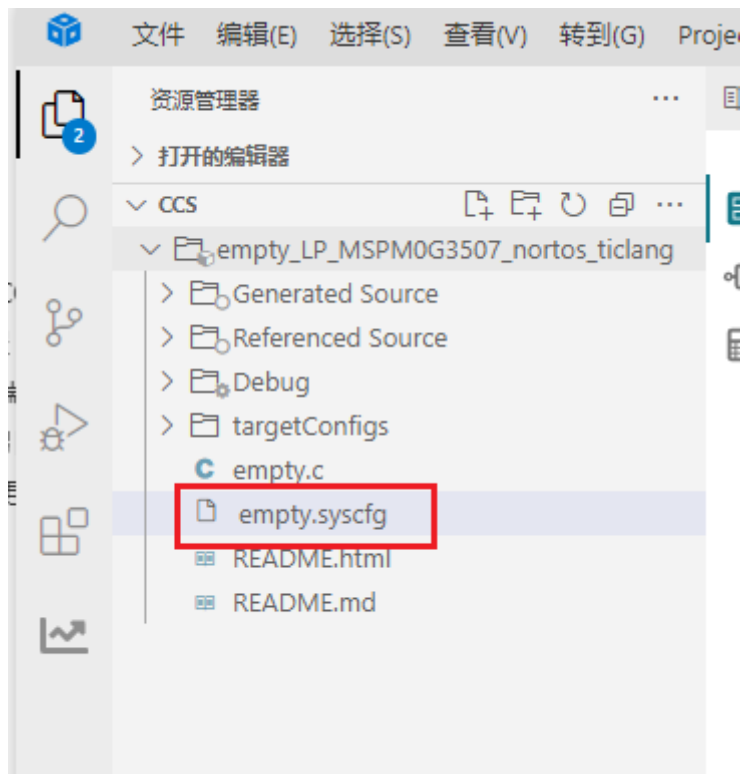
在单片机中，引脚的输入模式是指将某个引脚配置为输入功能，用于接收外部信号的变化。而按键的工作原理正是基于这一点：通过设置按键接入的引脚为输入模式，单片机可以检测引脚上的电平变化。当按键未按下时，引脚电平保持在低电平状态；当按键按下时，引脚电平变为高电平。这种高低电平的变化正是按键输入信号的体现，也正是单片机判断按键状态的依据。因此，接收外部信号的过程实际上就是通过输入模式检测按键高低电平变化的过程。

1. 打开SYSCONFIG配置工具

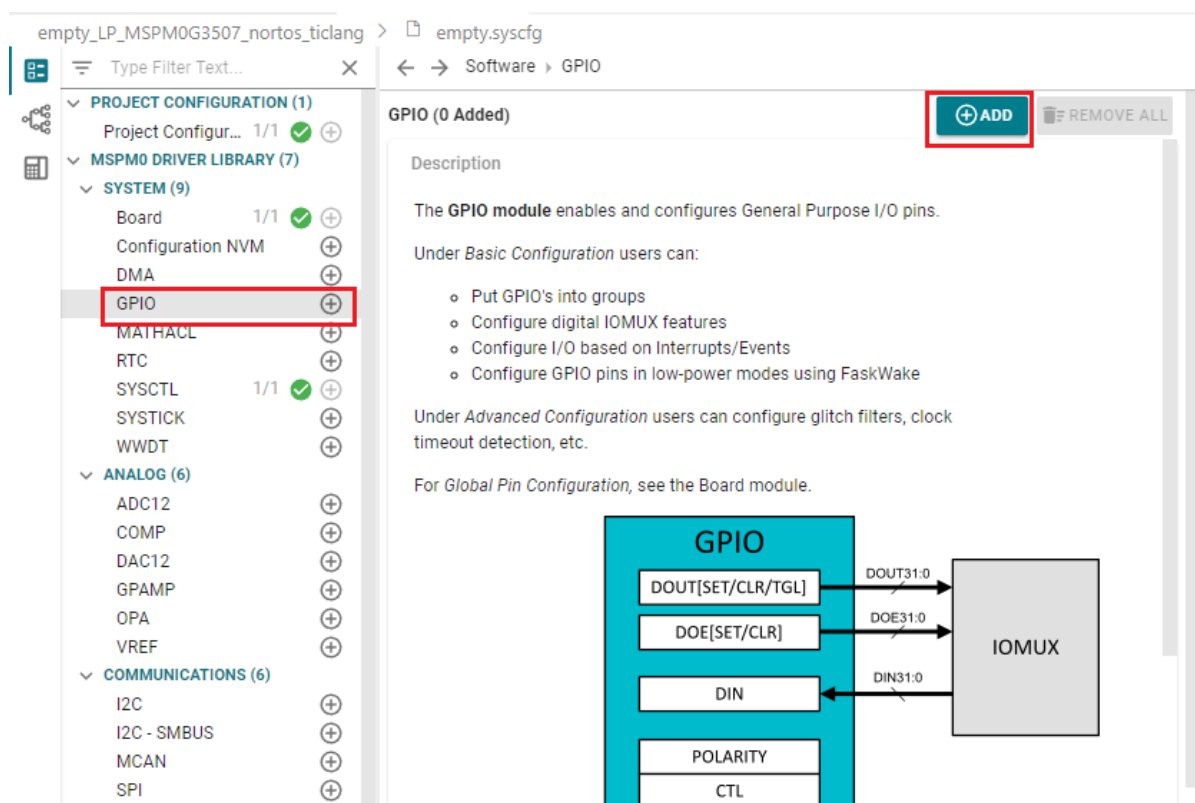
在CCS中新建一个空白工程 empty。



在CCS的左侧工作区中找到并打开empty.syscfg文件。



在 SYSCONFIG 中，左侧选择 MCU 外设，找到 GPIO 选项并点击进入。在 GPIO 中点击 ADD，添加两组引脚外设，一组LED引脚，一组按键引脚。LED引脚参考前面 点亮LED 的课程，这里只讲按键引脚的配置。



2. 配置引脚模式

配置引脚参数，将GPIO实例命名为 `KEY`，由于接入的是 PA18，所以 Port 选择 `PORTA`。

引脚名称设置为 `PIN_18`，设置为输入模式 `Input`。Internal Resistor设置为下拉电阻 `Pull-Down Resistor`，Assigned Pin设置要控制的引脚为18。

✓ KEY

Name	KEY
Port	PORTA
Port Segment	Any

Group Pins

1 added

+

 ADD

REMOVE ALL

✓ PIN_18

Name	PIN_18
Direction	Input
IO Structure [?]	Any

Digital IOMUX Features

Internal Resistor	Pull-Down Resistor
Invert	Disabled
Input Filter	Disabled
Enable Fast-Wake	☐
Override Pin Hysteresis Default	☐
Hysteresis Control	Disabled
Wakeup Logic	Disabled

Assigned Port	PORTA
Assigned Port Segment	Any
Assigned Pin	18

Interrupts/Events

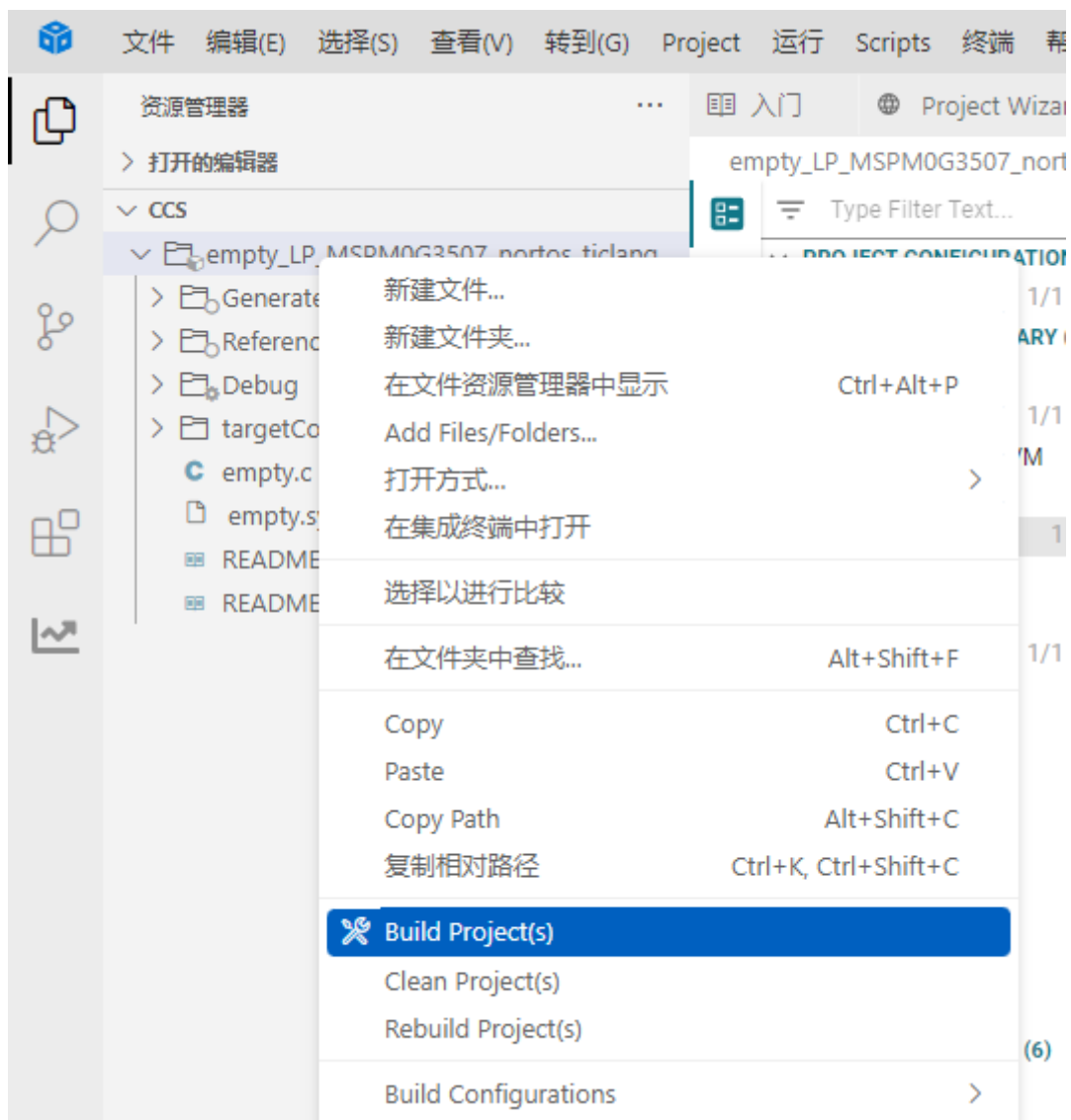
LaunchPad-Specific Pin	No Shortcut Used
------------------------	------------------

PinMux Peripheral and Pin Configuration

PIN_18	Any(PA18/11)
--------	--------------

通过快捷键 Ctrl+S 将.syscfg文件中的配置保存。

保存后编译



3. 写入程序

在empty.c文件中，写入以下代码

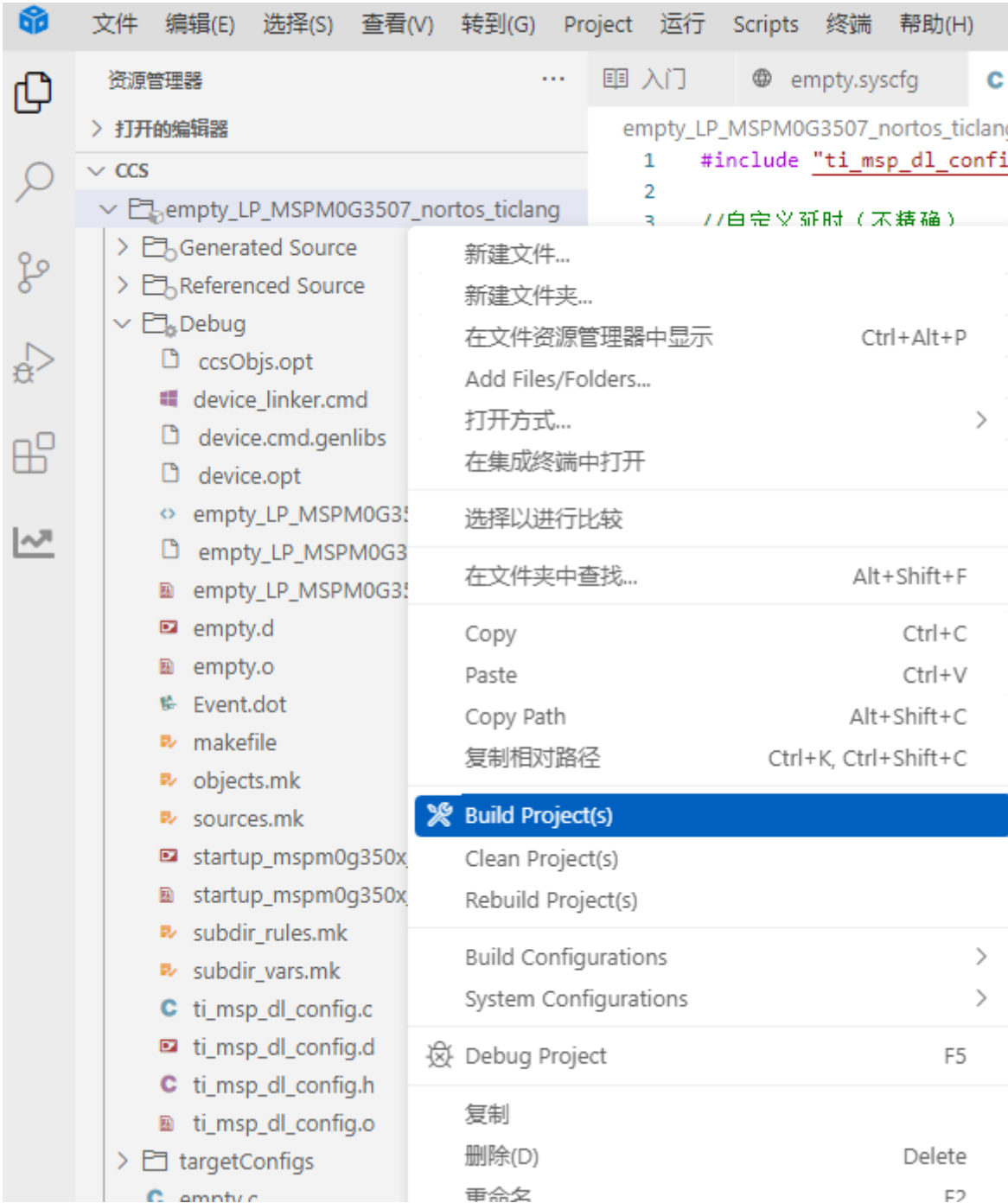
```
#include "ti_msp_dl_config.h"

int main(void)
{
    SYSCFG_DL_init();

    while (1)
    {
        //如果读取到的引脚值大于0，说明PA18引脚为高电平
        // If the pin value read is greater than 0, it means that the
        PA18 pin is high level.
        if( DL_GPIO_readPins(KEY_PORT, KEY_PIN_18_PIN) > 0 )
        {
            DL_GPIO_setPins(LED1_PORT, LED1_PIN_2_PIN); //LED控制输出
            高电平 LED control output high level
        }
        else//如果PA18引脚为低电平 If the PA18 pin is low
        {
            DL_GPIO_clearPins(LED1_PORT, LED1_PIN_2_PIN); //LED控制输出
            低电平 LED control output low level
        }
    }
}
```

```
    }  
}  
}
```

4. 编译



编译成功了，即可将程序下载到开发板。

四、程序分析

- empty.c

```

03_key > C empty.c > main
1  #include "ti_msp_dl_config.h"
2
3  int main(void)
4  {
5
6      SYSCFG_DL_init();
7
8      while (1)
9      {
10         //如果读取到的引脚值大于0, 说明PA18引脚为高电平
11         // If the pin value read is greater than 0, it means that the PA18 pin is high level.
12         if( DL_GPIO_readPins(KEY_PORT, KEY_PIN_18_PIN) > 0 )
13         {
14             DL_GPIO_setPins(LED1_PORT,LED1_PIN_2_PIN); //LED控制输出高电平 LED control output high level
15         }
16         else//如果PA18引脚为低电平 If the PA18 pin is low
17         {
18             DL_GPIO_clearPins(LED1_PORT,LED1_PIN_2_PIN); //LED控制输出低电平 LED control output low level
19         }
20     }
21 }

```

uint32_t DL_GPIO_readPins(GPIO_Regs* gpio, uint32_t pins) 用于读取指定 GPIO 引脚的电平状态。通过读取 GPIO 寄存器的值，获取指定引脚的状态。输入需要读取的 GPIO 端口和引脚号，返回该引脚的电平状态。

gpio: 读取的GPIO端口。

pins: 读取的引脚号。

这段代码通过不断检测引脚 **PA18**（按键接入引脚）的电平状态，控制 **LED1_PIN_2**（LED控制引脚）的输出电平，实现按键控制LED的功能。当 **PA18** 为高电平时，点亮 LED（LED 引脚输出高电平）；当 **PA18** 为低电平时，熄灭 LED（LED 引脚输出低电平）。

五、实验现象

程序下载完成之后，可以通过按键控制led灯的亮灭，按下按键，LED灯亮，松开按键，LED灯灭。

